# CI/CD architecture in DevOps — step-by-step 🚀

Short answer first: CI/CD (Continuous Integration / Continuous Delivery or Deployment) is an automated pipeline that takes code from a developer's Git commit to a running, tested, monitored release — fast, repeatable, and safe.

Below is a practical, step-by-step architecture you can implement or explain in an interview.

---

## Step-by-step CI/CD flow

1. **Source control (Git)**
   - Developer pushes code to branches (feature/bugfix/main).
   - Common tools: GitHub / GitLab / Bitbucket.
   - Best practice: pipeline-as-code (e.g., `.github/workflows/*`, `Jenkinsfile`).
2. **Trigger (events)**
   - Pipeline starts on events: push, PR open/merge, tag, schedule.
   - Use protected branches and required status checks.
3. **Pre-commit & PR checks**
   - Pre-commit hooks, linters, and lightweight tests run locally/CI to catch quick issues.
4. **Continuous Integration (build)**
   - Checkout code → install deps → compile/build artifacts.
   - Tools: Jenkins/GitHub Actions/GitLab CI/CircleCI.
   - Gate: build must succeed before moving forward.
5. **Automated unit tests**
   - Run unit tests; fail fast if coverage or tests fail.
   - Report test results and coverage.
6. **Static analysis & security scans**
   - Linting, SAST (static code analysis), dependency scanning (SCA).
   - Tools: SonarQube, ESLint, Bandit, Snyk, Dependabot.
7. **Package / Create artifact**
   - Produce immutable artifacts: JAR/WAR, ZIP, Docker image (tagged with commit SHA).
   - Tag artifacts with version + commit hash.
8. **Artifact repository / Registry**
   - Push to artifact store: Artifactory / Nexus / ECR / Docker Hub.
   - Immutable and versioned artifacts for traceability.
9. **Integration / acceptance tests in ephemeral env**
   - Deploy artifact to ephemeral or staging environment (containers or ephemeral VM).
   - Run integration, API, end-to-end tests, automated acceptance tests.
10. **Security & compliance gates**
    - Run DAST (dynamic tests), license checks, container image scans (Trivy/Clair).

- Block promotion if high/severe findings.

11. **Continuous Delivery (deploy) → staging + manual approvals**
    - If tests pass, promote to staging automatically; optionally require manual approval for production.
    - Infrastructure provisioning via IaC (Terraform / CloudFormation / Pulumi) at this stage.
12. **Continuous Deployment (production)**
    - Deploy to production using chosen strategy (rolling, blue-green, canary, immutable).
    - Deployment orchestrators: Kubernetes, AWS ECS, serverless, or traditional VM orchestration + config mgmt (Ansible).
13. **Smoke tests & automated verification**
    - Post-deploy smoke and health checks. Auto-verify endpoints, basic transactions.
14. **Observability & monitoring**
    - Metrics, logs, traces (Prometheus + Grafana, ELK/Opensearch, Jaeger, Datadog).
    - Set up alerts and SLOs/SLIs.
15. **Rollback & remediation**
    - If verification fails or alerts fire, automated rollback or rollback playbook triggers.
    - Ensure database migrations are reversible or use backward-compatible migrations.
16. **Audit, traceability & feedback loop**
    - Store pipeline logs, audit who promoted what and when. Feed incidents back to dev for fixes.

---

# Simple ASCII flow

```
Git (PR/Push)
   ↓
CI Server (build, unit tests, static analysis)
   ↓
Package (artifact/docker image) → Artifact Registry
   ↓
Deploy to Ephemeral/Staging → Integration & Security Tests
   ↓
Approval (optional)
   ↓
Deploy to Prod (canary/blue-green/rolling)
   ↓
Monitoring → Alerts → Rollback if needed
```

---

# Common deployment strategies (short)

- **Rolling:** update small sets of instances gradually.
- **Blue-Green:** switch traffic between two full environments.
- **Canary:** release to small % of users, analyze, then ramp up.

- **Immutable:** replace instances entirely (no in-place changes).

---

## Tools (examples by stage)

- SCM: **GitHub / GitLab / Bitbucket**
- CI: **Jenkins / GitHub Actions / GitLab CI / CircleCI**
- Build & Test: language build tools (Maven, Gradle, npm), test frameworks
- Artifacts: **Artifactory / Nexus / Docker Registry / ECR**
- IaC: **Terraform / CloudFormation / Pulumi**
- Orchestration: **Kubernetes / ECS / Ansible**
- Security: **Snyk / Trivy / Clair / SonarQube**
- Observability: **Prometheus/Grafana, ELK, Datadog, New Relic**

---

## Best practices checklist

- Pipeline-as-code and versioned pipelines.
- Immutable artifacts with commit hash tags.
- Environment parity (dev ≈ staging ≈ prod).
- Automated tests at all levels (unit → integration → e2e).
- Security scans integrated early (shift-left).
- Automated rollbacks + runbooks.
- Use feature flags for risky features.
- Keep secrets in a secrets manager (Vault, AWS Secrets Manager).
- Protect main branches and require code review.

---

## Quick example — minimal GitHub Actions CI (build + test)

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with: { 'java-version': '17' }
      - name: Build
        run: mvn -B package --file pom.xml
      - name: Run tests
        run: mvn test
      - name: Publish artifact
        # push to registry/artifactory step here
        run: echo "publish step"
```

---

If you want, I can:

- Convert this into a **diagram** or a one-page cheat sheet, or
- Build a **full pipeline example** for **Jenkins**, **GitHub Actions → Docker → Kubernetes**, or **Terraform + GitLab CI** (pick one) with real YAML/Jenkinsfile + IaC.

---

**DevOps CI full architecture step by step** (only Continuous Integration part, not full CD). Let's break this down as it would be explained in an **interview** or **real-world setup**.

---

# 🔄 Continuous Integration (CI) Architecture — Step by Step

CI ensures **every code change is built, tested, scanned, and validated automatically** before merging into the main branch.

---

## 1. Source Code Management (SCM)

- Developers push code to Git (GitHub, GitLab, Bitbucket).
- **Branching strategy**: feature → develop → main.
- Pipeline is defined as code (`Jenkinsfile`, `.gitlab-ci.yml`, `.github/workflows`).

---

## 2. Pipeline Trigger

- CI pipeline starts on:
  - Code push or Pull/Merge Request.
  - Scheduled (nightly builds).
  - Tag creation for releases.

---

## 3. Build Stage

- Pipeline runner/agent spins up.
- Pulls latest code.
- Installs dependencies.
- Compiles/builds project:
  - **Java** → `mvn package`

- **Node.js** → `npm install && npm run build`
- **Python** → `pip install -r requirements.txt`

---

# 4. Code Quality & Static Analysis

- Linting: `eslint`, `flake8`, `checkstyle`.
- Code style enforcement: `black`, `prettier`.
- Static code analysis: **SonarQube**, **CodeQL**.

---

# 5. Unit Testing

- Run automated unit tests.
- Generate test coverage reports.
- Fail pipeline if coverage < threshold.

Example:

```
pytest --cov=src tests/      # Python
mvn test                     # Java
npm test                     # Node.js
```

---

# 6. Security & Vulnerability Scans

- Dependency checks → `safety`, `npm audit`, `mvn dependency-check`.
- Secret detection → `trufflehog`, `git-secrets`.
- SAST (Static App Security Testing) → SonarQube, Bandit, CodeQL.

---

# 7. Build Artifact Creation

- Package build output:
  - Java → JAR/WAR.
  - Python → Wheel/Docker image.
  - Node.js → Docker image or bundle.

---

# 8. Artifact Repository

- Push artifacts to repository:
  - **Nexus / Artifactory** for JARs, Wheels.
  - **DockerHub / ECR / GCR** for Docker images.

---

## 9. Reports & Notifications

- Generate:
  - Build status.
  - Test reports.
  - Security reports.
- Notify developers via Slack/Teams/Email.

---

# 📊 CI Flow (Simplified)

```
Developer Push Code → Git Repo
          ↓
CI Tool (Jenkins / GitHub Actions / GitLab CI)
          ↓
Checkout Code → Install Dependencies
          ↓
Lint & Static Code Analysis
          ↓
Unit & Integration Tests
          ↓
Security & Vulnerability Scans
          ↓
Build Artifact (JAR/Docker/ZIP)
          ↓
Push to Artifact Repository
          ↓
Report & Notify Developers
```

---

# ✅ Best Practices for CI

- Keep builds fast (parallel jobs, caching).
- Fail fast (stop on first failure).
- Immutable artifacts (tag with commit SHA).
- Shift-left security (scan early in pipeline).
- Use **pipeline-as-code**.
- Run CI on **every PR** before merge.

---

Now let's cover **DevOps CD (Continuous Delivery / Continuous Deployment) full architecture step by step**.

☞ **Remember:**

- **Continuous Delivery (CD)** → automated delivery up to **staging**, production requires **manual approval**.

- **Continuous Deployment (CD)** → fully automated **deployment to production** without manual steps.

---

# 🚀 DevOps CD Architecture — Step by Step

---

## 1. Input: Tested Artifact from CI

- CD starts with an **artifact built by CI**:
  - JAR/WAR (Java),
  - Wheel (Python),
  - Docker image,
  - Helm Chart (Kubernetes).
- Artifact is versioned (commit SHA, tag) and stored in **Artifact Repository** (Nexus, Artifactory, DockerHub, ECR, etc.).

---

## 2. Deployment Pipeline Trigger

- Triggered by:
  - Merge to `main` branch.
  - New release tag.
  - Scheduled release.
  - Manual approval (for production).

---

## 3. Infrastructure Provisioning

- CD ensures infra exists before deployment:
  - **Terraform / CloudFormation / Pulumi** → cloud resources (VMs, K8s clusters, networks).
  - **Ansible / Chef / Puppet** → server configuration.
  - **Helm / Kustomize** → Kubernetes manifests.

---

## 4. Deploy to Test / QA Environment

- Pull artifact from repo.
- Deploy to **Test/QA/Staging environment**.
- Tools: Jenkins, GitHub Actions, ArgoCD, Spinnaker, GitLab CI.

---

# 5. Automated Tests in Staging

- Run **integration tests** → validate services talk correctly.
- Run **end-to-end (E2E) tests** → simulate user flows.
- Run **API tests / contract tests**.
- Fail pipeline if issues found.

---

# 6. Security & Compliance Checks

- Container image scan (Trivy, Clair).
- DAST (Dynamic Application Security Testing).
- Compliance checks (licenses, configs).
- Policy enforcement (OPA, Checkov).

---

# 7. Promotion to Production

- **Continuous Delivery** → requires manual approval before production.
- **Continuous Deployment** → automatically promotes to production.
- Artifact version promoted, not rebuilt (immutability).

---

# 8. Deployment Strategies

Different strategies used in CD:

- **Blue-Green Deployment**
  - Two environments: Blue (live), Green (idle).
  - Deploy to Green, run tests → switch traffic.
- **Canary Deployment**
  - Release to small % of users (e.g., 5%).
  - Monitor metrics → gradually increase rollout.
- **Rolling Update**
  - Gradually replace instances/pods with new version.
- **Immutable Deployment**
  - Replace old servers/containers completely.

---

# 9. Post-Deployment Verification

- **Smoke tests** → check health endpoints.
- **Monitoring validation**: CPU, memory, errors, latency.
- **Business KPI monitoring**: signups, transactions.

---

## 10. Observability & Feedback Loop

- **Monitoring:** Prometheus + Grafana, CloudWatch, Datadog.
- **Logging:** ELK / OpenSearch.
- **Tracing:** Jaeger, OpenTelemetry.
- **Alerting:** Slack, PagerDuty, Opsgenie.
- Feed issues back to developers automatically.

---

## 11. Rollback Strategy

- If failure:
  - Rollback to last stable artifact.
  - Switch back to Blue env (Blue-Green).
  - Stop canary rollout.
  - GitOps tools (ArgoCD, Flux) auto-revert Git state.

---

# 📊 CD Flow (Simplified)

```
Artifact (from CI) → Artifact Repo
          ↓
Infra Provisioning (Terraform/Ansible/Helm)
          ↓
Deploy to Staging → Integration & E2E Tests
          ↓
Security & Compliance Checks
          ↓
Approval (optional)
          ↓
Deploy to Production (Blue-Green / Canary / Rolling)
          ↓
Smoke Tests + Monitoring
          ↓
Rollback if Failure → Feedback to Devs
```

---

# ✅ Best Practices in CD

- Immutable artifacts (no rebuilds after CI).
- Standardized environments (Dev ≈ Staging ≈ Prod).

- Automated rollback & disaster recovery plans.
- Feature flags for risky changes.
- Secrets managed with Vault / AWS Secrets Manager.
- GitOps for Kubernetes (ArgoCD, Flux).

---

⚡ In short:

- **CI** = Build, Test, Package.
- **CD** = Deploy, Verify, Monitor, Rollback.

---

**CI/CD (Continuous Integration and Continuous Deployment/Delivery) Architecture** in a clear way.

---

# ◆ What is CI/CD?

- **CI (Continuous Integration)** → Developers frequently merge code changes into a shared repo. Automated builds and tests run to detect issues early.
- **CD (Continuous Delivery/Deployment)** → Once code passes tests, it's automatically packaged and deployed to staging or production environments.

☞ Together, CI/CD ensures **faster, reliable, and automated software delivery**.

---

# ◆ CI/CD Architecture (Step by Step)

### 1. Source Code Management (SCM)

- Developers push code to a version control system like **GitHub, GitLab, Bitbucket, or Azure Repos**.
- Every commit or pull request triggers the CI/CD pipeline.
- 🔹 Tools: Git, GitHub, GitLab, Bitbucket.

---

### 2. CI Server (Build & Integration Stage)

- CI tool detects new commits.
- Pulls the latest code from repo.
- Compiles/builds the code (e.g., Java → JAR/WAR, .NET → DLL, NodeJS → bundle).

- Runs **unit tests**, **lint checks**, **code quality scans**.
- 🔻 Tools: Jenkins, GitHub Actions, GitLab CI, Azure DevOps, CircleCI.

---

### 3. Artifact Repository

- After successful build, the artifacts are stored in a **binary repo**.
- Examples:
  - Java → JAR/WAR to **Nexus / Artifactory**
  - Docker image → **Docker Hub / ECR / ACR / GCR**
- This ensures versioning & rollback capabilities.

---

### 4. CD Pipeline (Deployment Stage)

- Once artifacts are ready, deployment begins.
- Pipeline automates:
  - **Provisioning** (Terraform, Ansible, CloudFormation).
  - **Configuration Management** (Ansible, Chef, Puppet).
  - **Containerization & Orchestration** (Docker, Kubernetes, ECS/EKS/AKS/GKE).
  - **Application Deployment** (Helm charts, kubectl, AWS CodeDeploy, ArgoCD).

---

### 5. Testing & Quality Gates

- Before production release, run:
  - **Integration tests**
  - **Functional tests**
  - **Load/performance tests**
  - **Security scans (DevSecOps)**
- Tools: Selenium, JUnit, SonarQube, OWASP Dependency Check.

---

### 6. Deployment Strategies

- **Blue-Green Deployment** → Two identical environments, switch traffic between them.
- **Canary Deployment** → Gradual rollout to small % of users.
- **Rolling Updates** → Update pods/instances gradually in Kubernetes.

---

### 7. Monitoring & Feedback

- Once deployed, monitoring tools track app health.

- Alerts & dashboards help teams act quickly.
- Tools: Prometheus, Grafana, DataDog, ELK Stack, CloudWatch, Splunk.

---

# ◆ CI/CD Architecture Diagram (Conceptual)

```
   [ Developer Commit ]
            |
            v
[ Source Control: GitHub/GitLab ]
            |
            v
[ CI Server: Jenkins/GitHub Actions ]
  - Build
  - Unit Tests
  - Code Scan
            |
            v
[ Artifact Repository: Nexus / Docker Registry ]
            |
            v
[ CD Pipeline ]
  - Infrastructure as Code (Terraform, Ansible)
  - Deploy to Staging
  - Automated Tests
  - Deploy to Prod
            |
            v
[ Monitoring & Feedback: Grafana, ELK, Prometheus ]
```

---

# ◆ Benefits of CI/CD Architecture

- Faster delivery of features.
- Early detection of bugs.
- Automated deployments reduce manual errors.
- Scalable (works across VMs, containers, cloud).
- Enables **DevOps culture** (collaboration between dev & ops).

---

✅ **In short:**
CI/CD Architecture = **Source Control → CI Server → Artifact Repo → CD Pipeline → Monitoring & Feedback**, automated end-to-end.